



Advanced NCCL Tutorial: Beyond Collectives

Jeff Hammond, Ke Wen, Hanyue He

NVIDIA

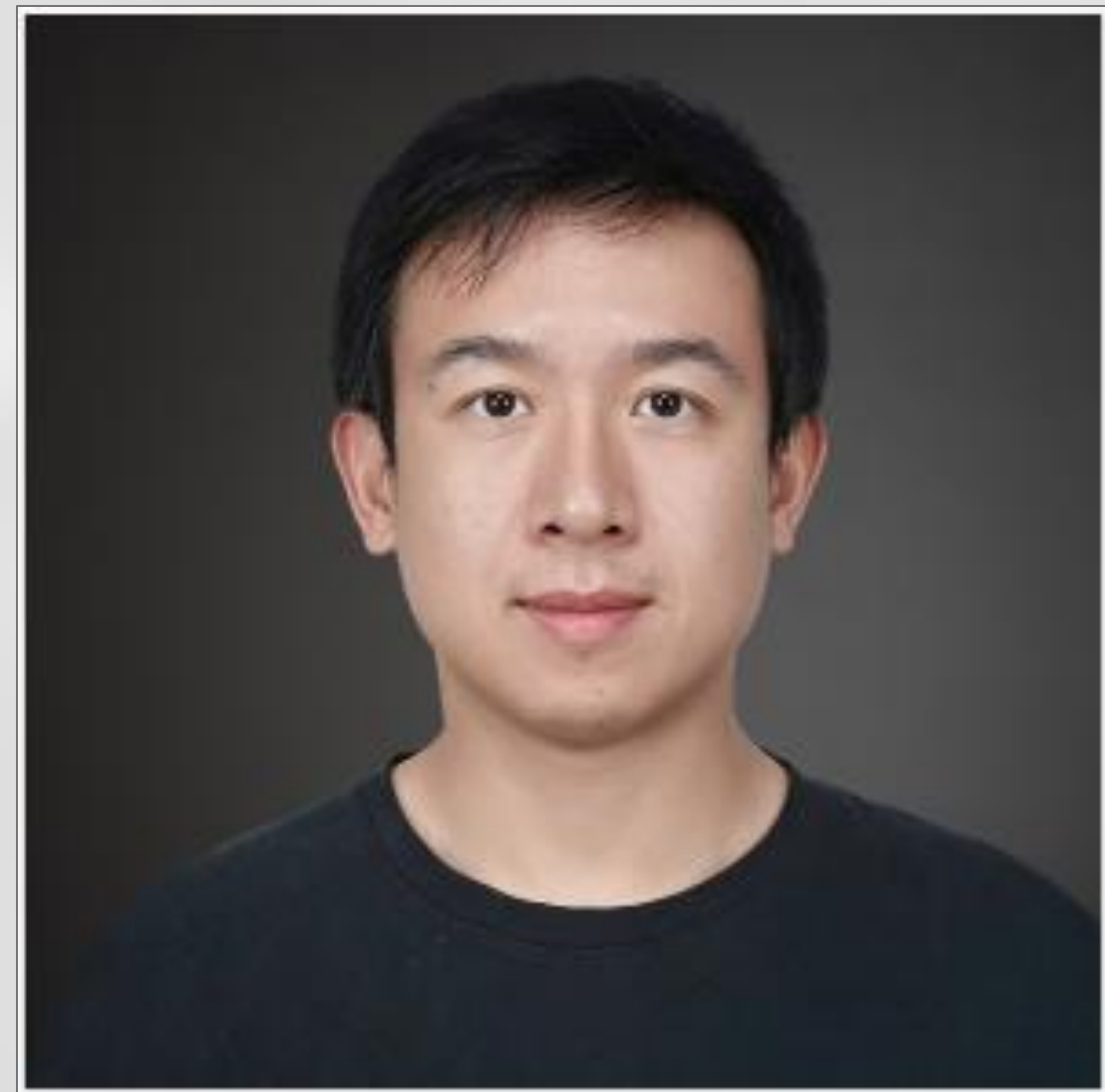
PyTorch China, 2026



Who are we?



Jeff Hammond
Distinguished Engineer
NVIDIA



Ke Wen
Principal Engineer
NVIDIA

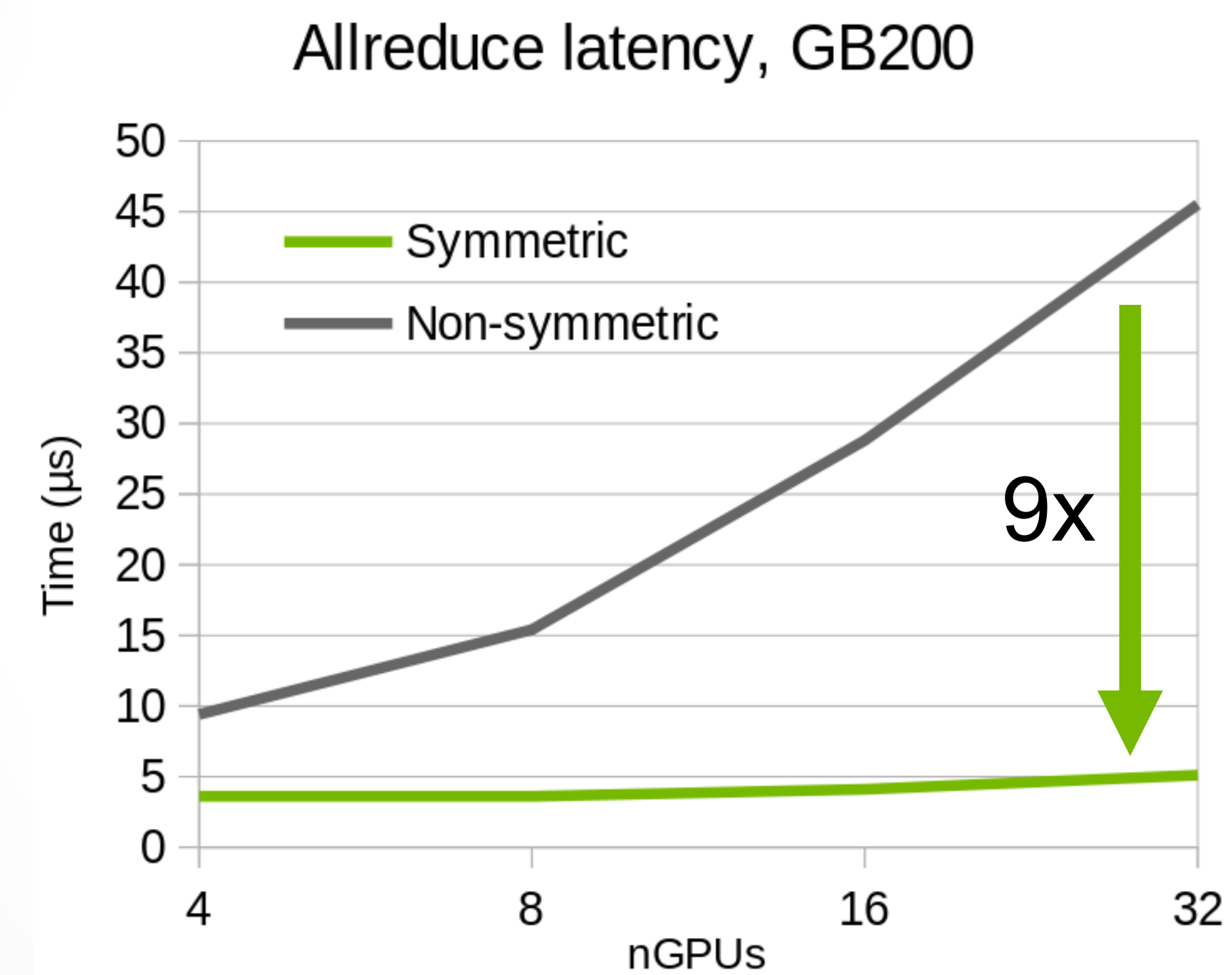


Hanyue He
Senior DevTech Engineer
NVIDIA



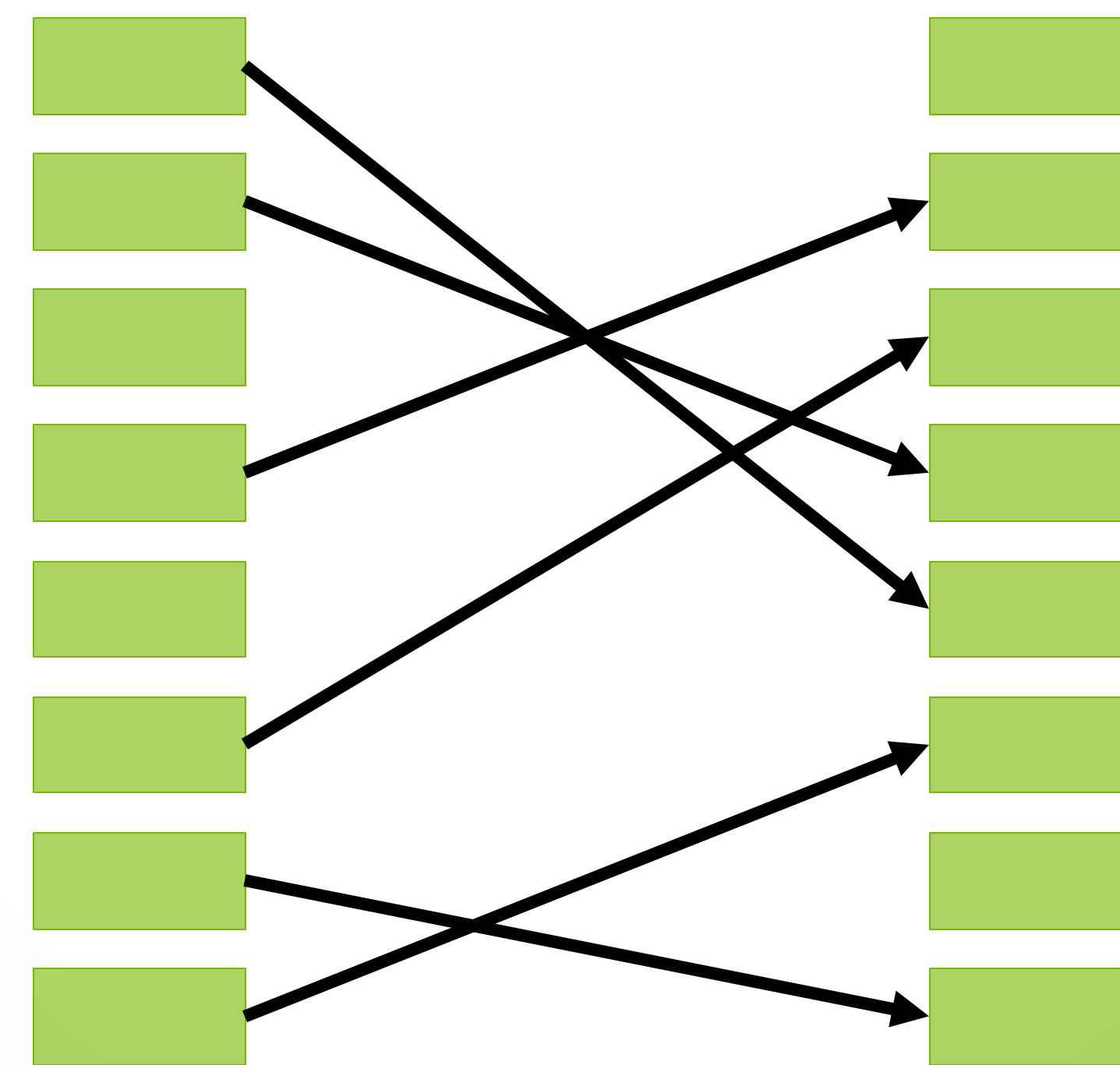
Today's Challenges in Distributed Workloads

Latency-sensitive operations (e.g. Inference)



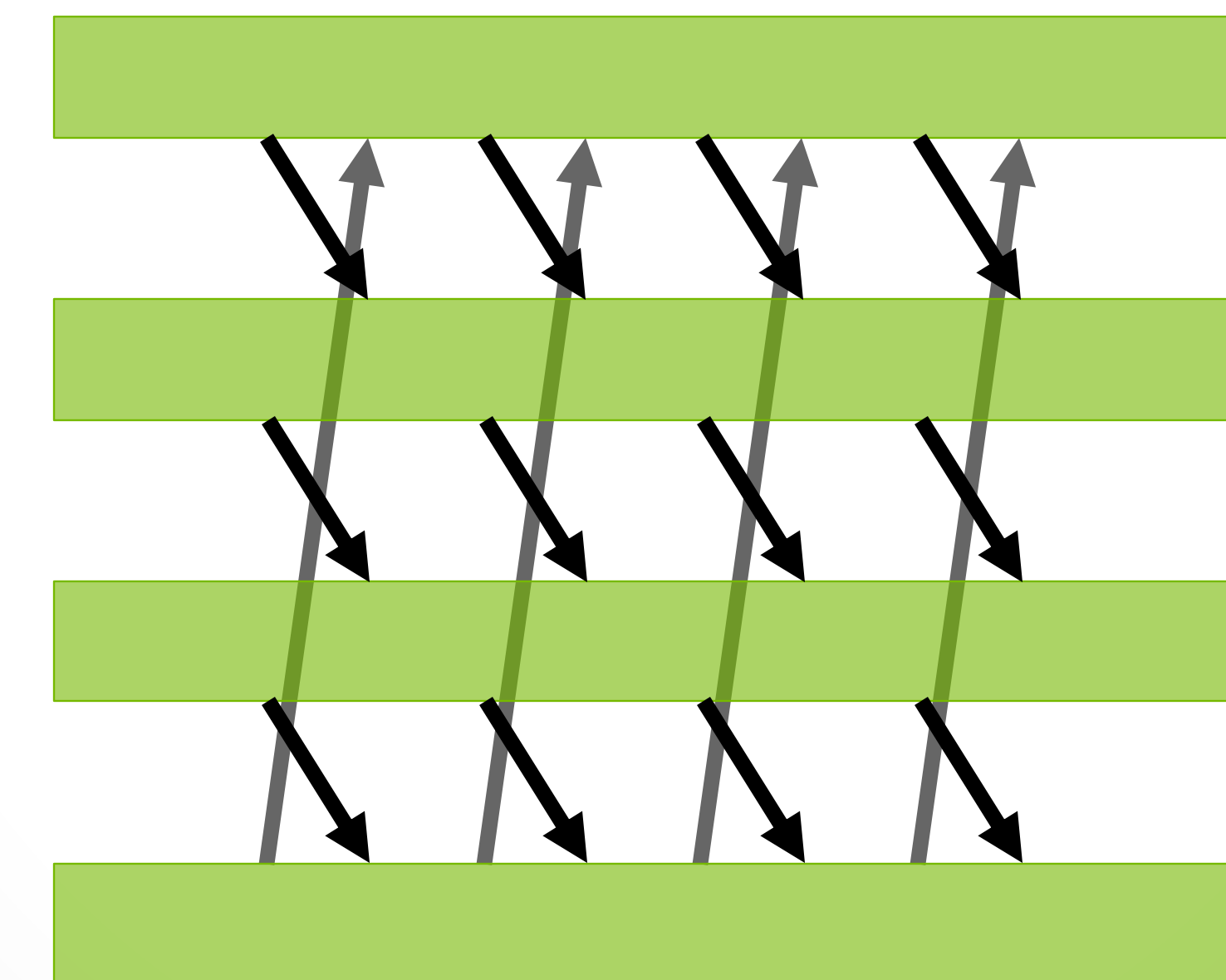
vLLM, SGLang, TRT-LLM

Custom comm patterns (e.g. MoE)



On-device metadata
(dest IDs, splits)

Comm-compute fusion In-kernel communication



Flash-infer, Helion

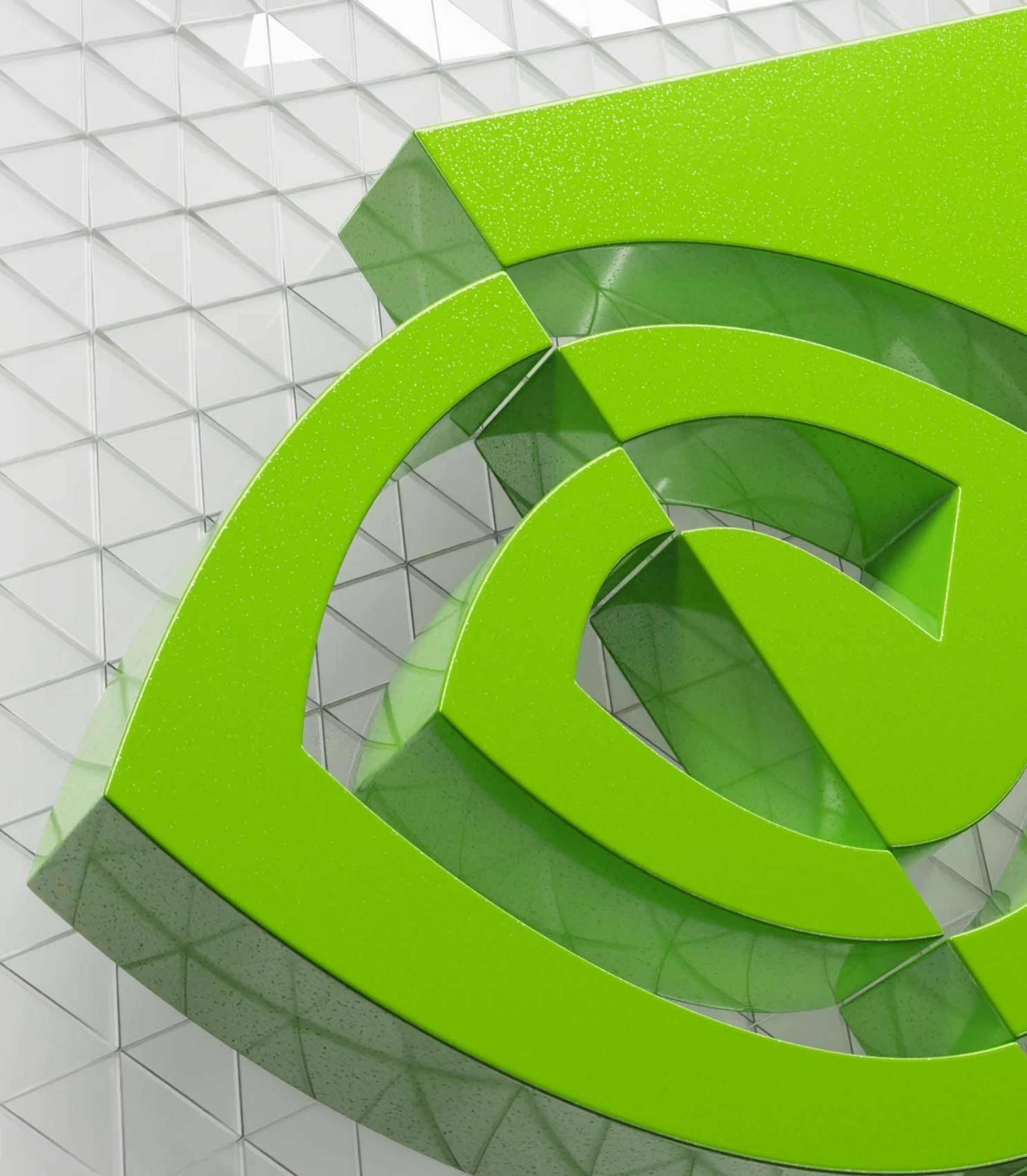
Agenda

NCCL Background

Device API Purpose

Symmetric Memory (LSA)

GPU-Initiated Networking (GIN)



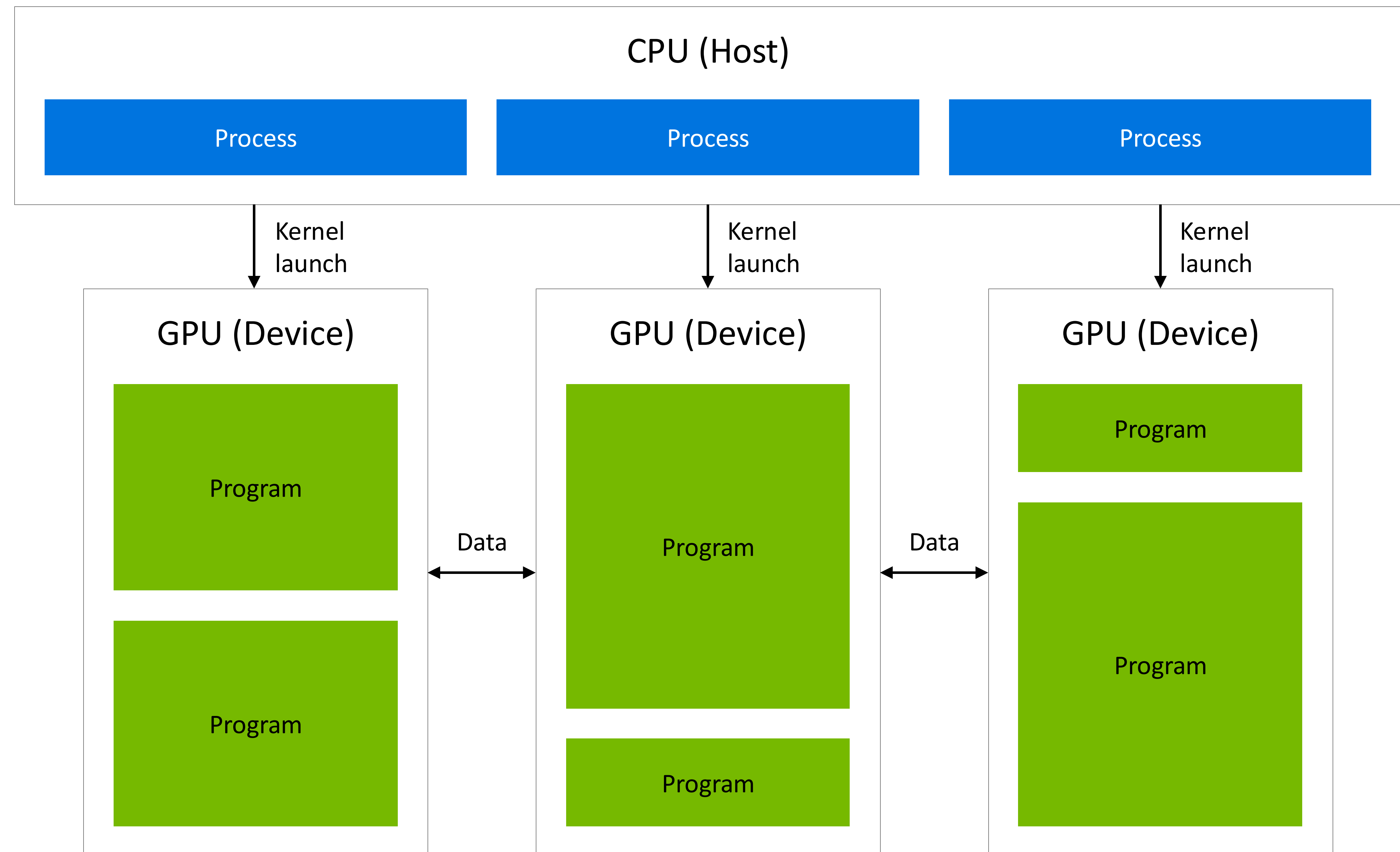
Multi-GPU Programs

Workloads are launched across **multiple GPUs** and managed by multiple processes (or threads).

When multiple processes participate in communication cooperatively, it's called a *collective*.

Data moves between GPUs via NVLINK, PCIe, InfiniBand, Ethernet, or other networks.

Combining computation and communication requires careful coordination to avoid deadlock.



NCCL Introduction



Optimized inter-GPU communication

NCCL: NVIDIA Collective Communication Library
Communication library running on GPUs, for GPU buffers.

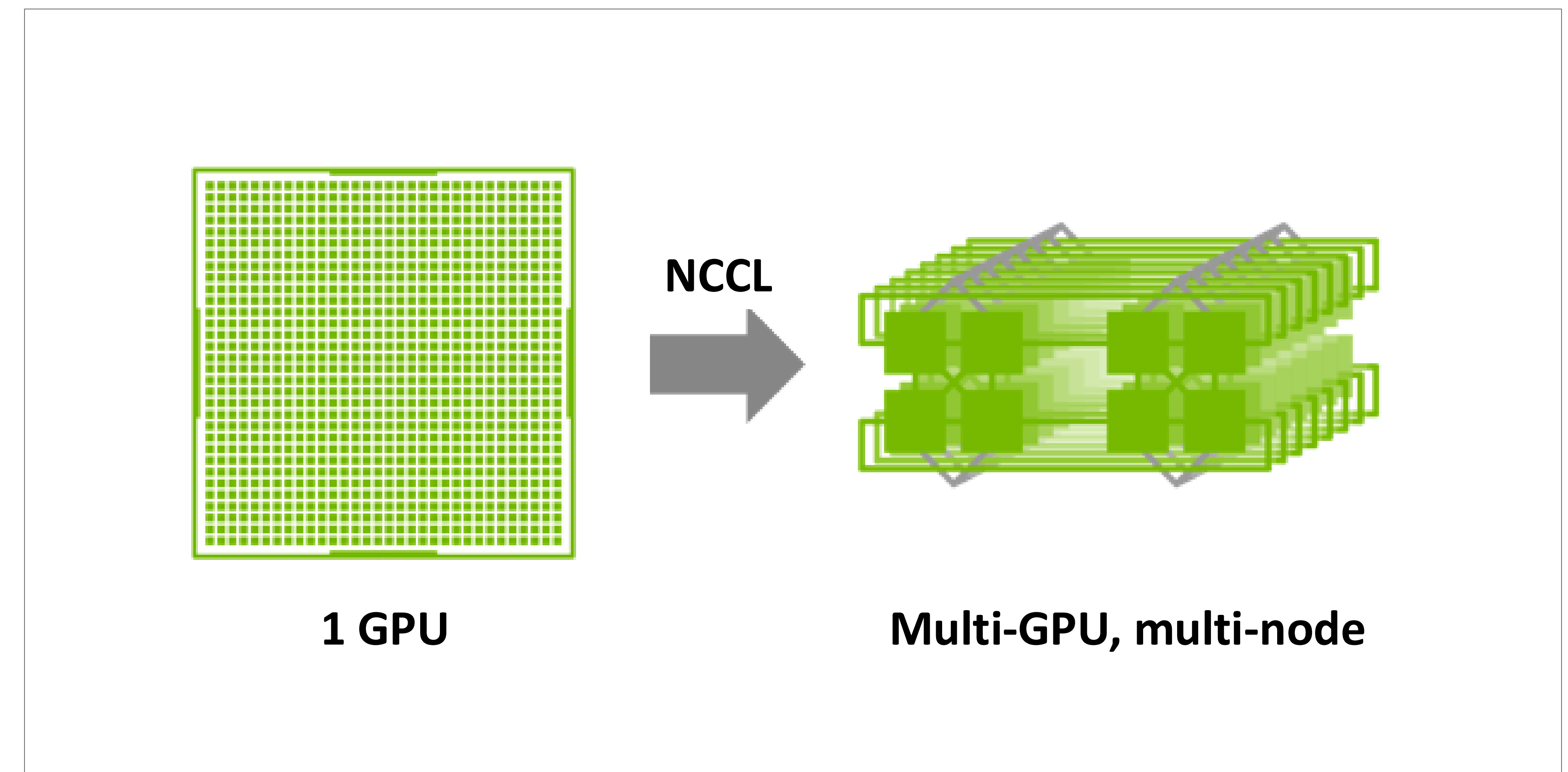
NCCL (pronounced “Nickel”) is a library developed by NVIDIA for efficient communication between multiple GPUs.

Supports single node and across multiple nodes.

Collective and point-to-point operations
(e.g., Allreduce, Broadcast, Send/Recv, Put).

Host Library: Communication calls are translated to a GPU kernel (running on a CUDA stream).

Device Library: Primitives for load-store remote access and network communication initiated within GPU kernels.



Binaries: <https://developer.nvidia.com/nccl> and in NGC containers

Source code: <https://github.com/nvidia/nccl>

Perf tests: <https://github.com/nvidia/nccl-tests>

NCCL Host APIs



NCCL APIs

Communication

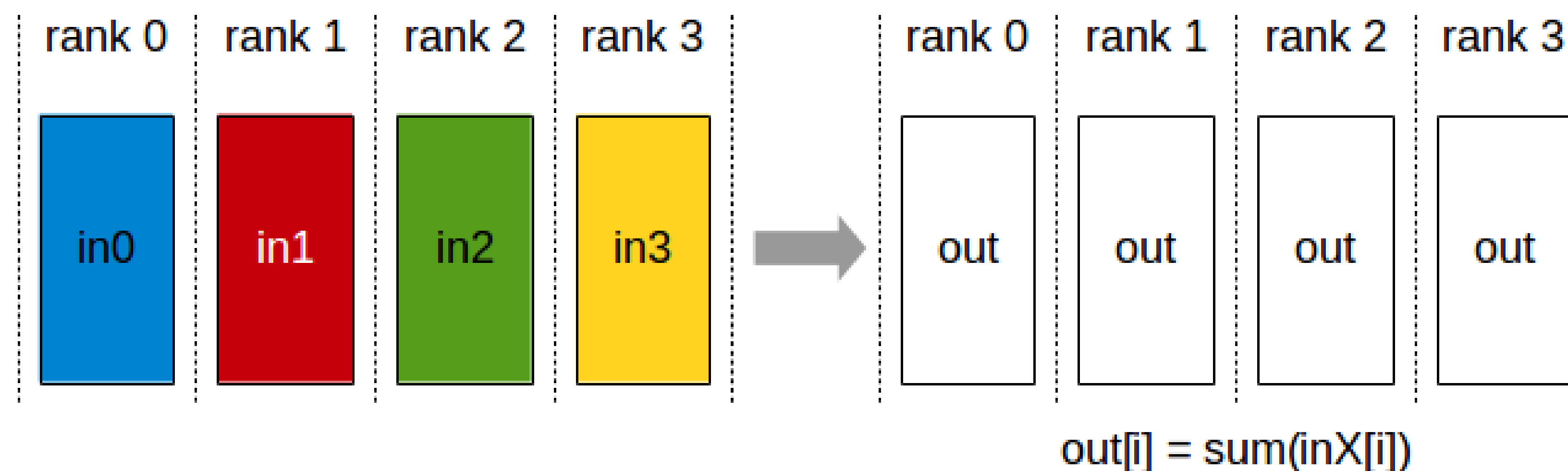
Send/recv

```
ncclSend(void* sbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);  
ncclRecv(void* rbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);
```

Collective operations

```
ncclAllReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op,  
              ncclComm_t comm, cudaStream_t stream);  
ncclBroadcast(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, int root,  
              ncclComm_t comm, cudaStream_t stream);  
ncclReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op, int root,  
           ncclComm_t comm, cudaStream_t stream);
```

Allreduce example



NCCL API

Send/receive with NCCL

```
...  
// Initialized NCCL communicator
```

```
int N=16;  
cudaStream_t stream;  
cudaStreamCreate(&stream);  
if (rank == 0) {  
    ncclSend(send_buf, N, ncclInt, 1, nccl_comm, stream);  
} else if (rank == 1) {  
    ncclRecv(recv_buf, N, ncclInt, 0, nccl_comm, stream);  
}
```

```
cudaStreamSynchronize(stream);
```

```
// Destroy NCCL communicator
```

```
...
```

Creating GPU stream for NCCL stream

Sending 16 * ncclInt to rank 1 as part of nccl_comm

NCCL API

Fused communication calls

Multiple calls to `ncclSend()` and `ncclRecv()` should be fused with `ncclGroupStart()` and `ncclGroupEnd()` to

Avoid deadlocks, e.g. if calls need to progress concurrently.

For more performance: fused operations can be more efficient by better utilizing the available IO.

Send/recv

```
ncclGroupStart();
ncclSend(sendbuff, sendcount, sendtype, peer, comm, stream);
ncclRecv(recvbuff, recvcount, recvtype, peer, comm, stream);
ncclGroupEnd();
```

Bcast

```
ncclGroupStart();
if (rank == root) {
    for (int r=0; r<n ranks; r++)
        ncclSend(sendbuff[r], size, type, r, comm, stream);
}
ncclRecv(recvbuff, size, type, root, comm, stream);
ncclGroupEnd();
```

NCCL Device APIs



NCCL Device API

In-kernel communication toolbox

Host-side API

Device Communicator

Designates a group of ranks (GPUs).
Device-side equivalent to the NCCL comm.
Created with `ncclDevCommCreate`.

Window

Designates buffers (memory).
Group of buffers registered across all ranks.
Created with `ncclCommWindowRegister`.

Coop

Designates GPU threads dedicated to
communication.

passed
to the kernel



Device-side API

LSA

Communication primitives for
Load/Store Accessible memory

Multimem

Communication primitives for NVLink SHARP

GIN

GPU-Initiated Networking primitives

Four objects tell the kernel who, where, and how

Host setup creates them; device code composes them

Team chooses ranks
Coop chooses threads

```
__global__ void kernel(..., ncclDevComm dev_comm, ncclWindow_t payload)
```

ncclDevComm rank, size, topology, and reserved device resources

ncclWindow_t registered allocation addressed by byte offset

ncclTeamWorld / **ncclTeamLsa** which ranks participate

ncclCoopThread / **ncclCoopWarp** / **ncclCoopCta** which GPU threads call together

NCCL Device API

In-kernel communication toolbox

LSA

Remote memory access

→ Get pointer for remote memory

→ Perform normal memory operations to pointer

Load/Store Accessible

Synchronization

→ LSA barrier

Multimem

NVLink SHARP access

→ Get pointer to multimem mappings

→ Perform multimem operations on pointer

NVLink SHARP

Synchronization

→ LSA barrier with Multimem scope

GIN

Remote memory access

→ Create GIN object, select GIN context

→ Perform put() operations

→ Wait on signals for remote operations

→ Wait on counters for local operations

GPU-Initiated Networking

Synchronization

→ GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {
```

```
float* ptr = ncclGetPeerPointer(devComm, offset, peer);  
ptr[0] = value;
```

```
ncclBarrierSession bar { ncclCoopCta(), ncclTeamTagLsa(),  
                        devComm, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

```
float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);  
value = multimem_reduce_sum(ptr);  
multimem_store(ptr, value);
```

```
ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };  
gin.put(ncclTeamWorld(devComm), peer,  
        win, remoteOffset, win, localOffset,  
        size, ncclGin_SignalInc{signalIndex});  
gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);
```

```
ncclBarrierSession bar {  
    ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,  
        ncclGinFenceLevel::relaxed);
```

```
}
```

NCCL Device API

In-kernel communication toolbox

LSA

Remote memory access

→Get pointer for remote memory

→Perform normal memory operations to pointer

Load/Store Accessible

Synchronization

→ LSA barrier

Multimem

NVLink SHARP access

→Get pointer to multimem mappings

→ Perform multimem operations on pointer

NVLink SHARP

Synchronization

→ LSA barrier with Multimem scope

GIN

Remote memory access

→Create GIN object, select GIN context

→Perform put() operations

→Wait on signals for remote operations

→Wait on counters for local operations

GPU-Initiated Networking

Synchronization

→GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {
```

```
float* ptr = ncclGetPeerPointer(devComm, offset, peer);  
ptr[0] = value;
```

```
ncclBarrierSession bar { ncclCoopCta(), ncclTeamTagLsa(),  
                        devComm, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

```
float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);  
value = multimem_reduce_sum(ptr);  
multimem_store(ptr, value);
```

```
ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };  
gin.put(ncclTeamWorld(devComm), peer,  
        win, remoteOffset, win, localOffset,  
        size, ncclGin_SignalInc{signalIndex});  
gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);
```

```
ncclBarrierSession bar {  
    ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,  
        ncclGinFenceLevel::relaxed);
```

```
}
```

NCCL Device API

In-kernel communication toolbox

LSA

Remote memory access

→Get pointer for remote memory

→Perform normal memory operations to pointer

Load/Store Accessible

Synchronization

→ LSA barrier

Multimem

NVLink SHARP access

→Get pointer to multimem mappings

→ Perform multimem operations on pointer

NVLink SHARP

Synchronization

→ LSA barrier with Multimem scope

GIN

Remote memory access

→Create GIN object, select GIN context

→Perform put() operations

→Wait on signals for remote operations

→Wait on counters for local operations

GPU-Initiated Networking

Synchronization

→GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {
```

```
    float* ptr = ncclGetPeerPointer(devComm, offset, peer);  
    ptr[0] = value;
```

```
    ncclBarrierSession bar { ncclCoopCta(), ncclTeamTagLsa(),  
                             devComm, blockIdx.x };  
    bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

```
    float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);  
    value = multimem_reduce_sum(ptr);  
    multimem_store(ptr, value);
```

```
    ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };  
    gin.put(ncclTeamWorld(devComm), peer,  
            win, remoteOffset, win, localOffset,  
            size, ncclGin_SignalInc{signalIndex});  
    gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);
```

```
    ncclBarrierSession bar {  
        ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };  
    bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,  
            ncclGinFenceLevel::relaxed);
```

```
}
```

NCCL Device API

In-kernel communication toolbox

LSA

Remote memory access

→ Get pointer for remote memory

→ Perform normal memory operations to pointer

Load/Store Accessible

Synchronization

→ LSA barrier

Multimem

NVLink SHARP access

→ Get pointer to multimem mappings

→ Perform multimem operations on pointer

NVLink SHARP

Synchronization

→ LSA barrier with Multimem scope

GIN

Remote memory access

→ Create GIN object, select GIN context

→ Perform put() operations

→ Wait on signals for remote operations

→ Wait on counters for local operations

GPU-Initiated Networking

Synchronization

→ GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {
```

```
float* ptr = ncclGetPeerPointer(devComm, offset, peer);  
ptr[0] = value;
```

```
ncclBarrierSession bar { ncclCoopCta(), ncclTeamTagLsa(),  
                          devComm, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

```
float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);  
value = multimem_reduce_sum(ptr);  
multimem_store(ptr, value);
```

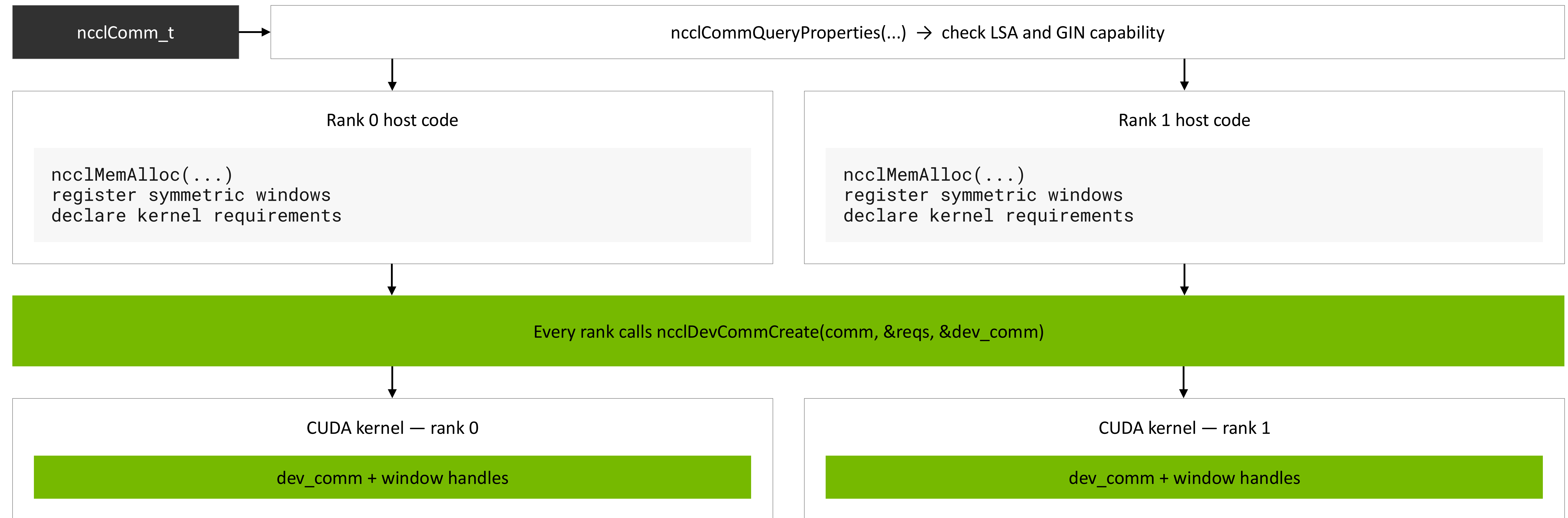
```
ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };  
gin.put(ncclTeamWorld(devComm), peer,  
        win, remoteOffset, win, localOffset,  
        size, ncclGin_SignalInc{signalIndex});  
gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);
```

```
ncclBarrierSession bar {  
    ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };  
bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,  
        ncclGinFenceLevel::relaxed);
```

```
}
```

The host prepares the objects the kernel will use

Setup is collective; Kernels receive a device communicator and window handles

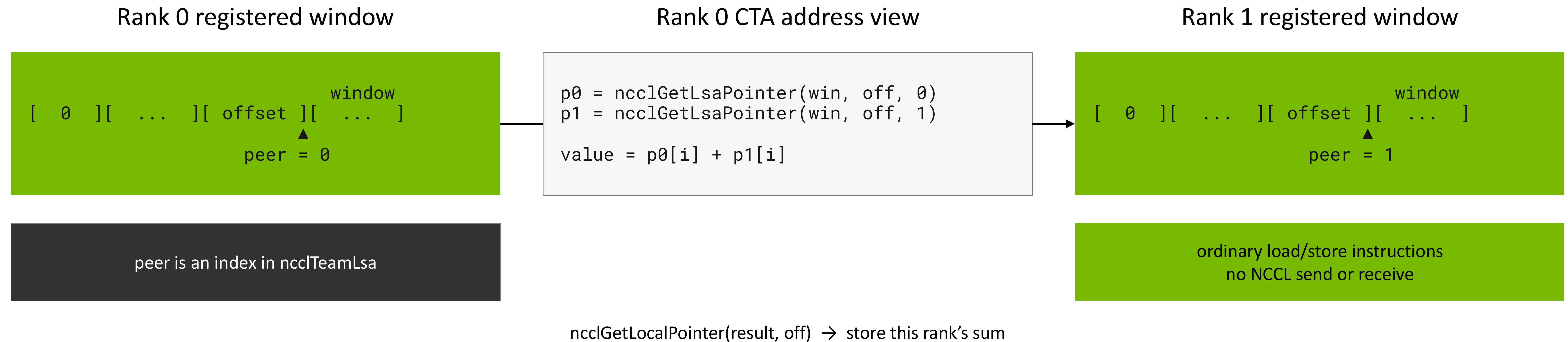


After stream completion: destroy dev comm → deregister windows → free memory

```
ncclDevCommRequirements reqs = NCCL_DEV_COMM_REQUIREMENTS_INITIALIZER;  
ncclDevCommCreate(comm, &reqs, &dev_comm);  
kernel<<<grid, block, 0, stream>>>(dev_comm, payload_window, ...);
```

LSA gives each CTA pointers into every peer's window

The window, byte offset, and LSA-team rank resolve to a locally accessible address



```
bar.sync(ncclCoopCta(), cuda::memory_order_acquire);  
  
auto *peer = ncclGetLsaPointer(source_window, byte_offset, lsa_peer);  
  
auto *local = ncclGetLocalPointer(result_window, byte_offset);  
  
bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

Exercise (Homework): LSA device API

Build a two-rank all-reduce with direct LSA loads

Inside one CTA, synchronize the LSA team, load both ranks' source windows, store the local sum, and synchronize again

https://github.com/NVIDIA/nccl/blob/master/docs/examples/06_device_api/01_allreduce_lsa/

```
ncclLsaBarrierSession<ncclCoopCta> bar{...}
```

Synchronizes participating CTAs across the LSA team.

```
ncclGetLsaPointer(source_window, 0, lsa_peer)
```

Returns a pointer the kernel can load directly.

```
ncclGetLocalPointer(result_window, 0)
```

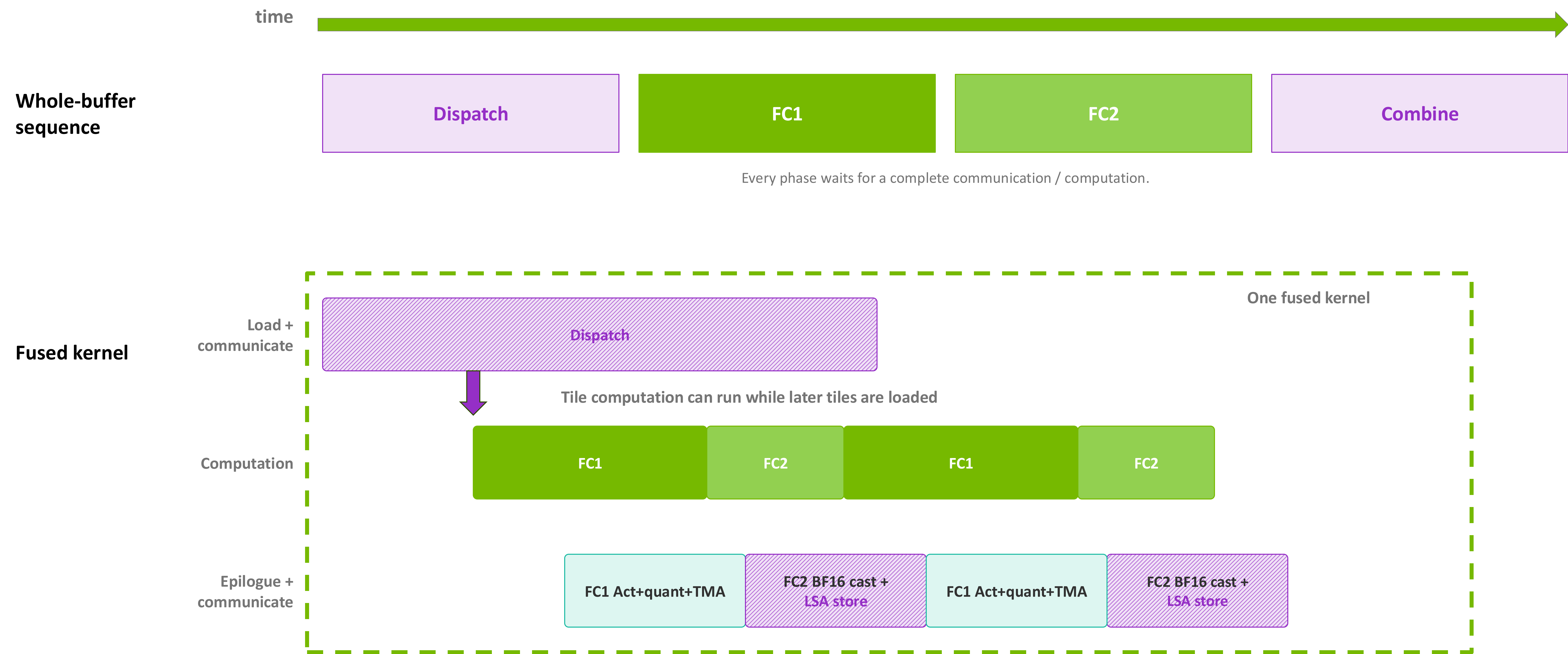
Returns this rank's registered result pointer; the release barrier closes the session.

MegaMoE
+ NCCL LSA



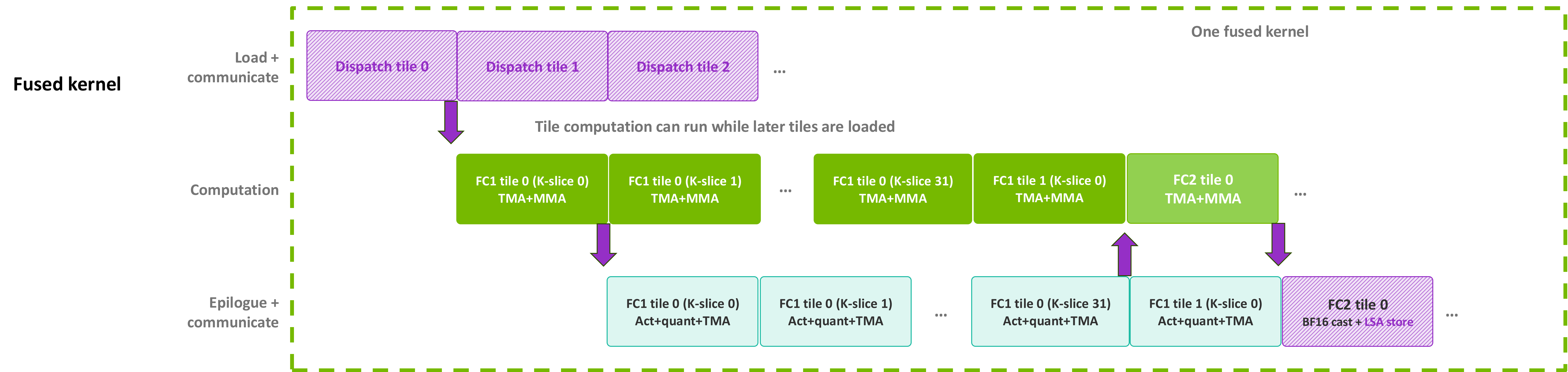
One kernel fuses dispatch, FC1, FC2, and combine

In a conventional whole-buffer sequence, communication and computation are sequential



Fusion starts computation tile by tile

A production kernel can compute completed tiles while communicating later tiles



NCCL LSA setup creates peer-addressable memory

Each EP rank registers the same arena layout once, then passes peer address offsets to the fused kernel

ONE-TIME HOST SETUP

- 1

Create the communicator

Rank 0 broadcasts the NCCL unique ID; every EP rank calls `ncclCommInitRank`.
- 2

Allocate equal-sized arenas

Each rank uses `ncclMemAlloc`. The runtime suballocates activations, scales, top-k weights and peer-visible workspace at identical offsets.
- 3

Register one symmetric window

All ranks collectively call `ncclCommWindowRegister` with `NCCL_WIN_COLL_SYMMETRIC`.
- 4

Resolve every peer alias

`ncclGetPeerDevicePointer` returns the device address used to access each rank's arena.

SAME LOGICAL LAYOUT ON EVERY RANK



WHAT THE KERNEL RECEIVES



NCCL establishes which local peers are Load/Store Accessible. The kernel receives peer virtual addresses, not a send/rcv schedule.

`ncclMemAlloc` · `ncclCommWindowRegister(..., NCCL_WIN_COLL_SYMMETRIC)` · `ncclGetPeerDevicePointer`

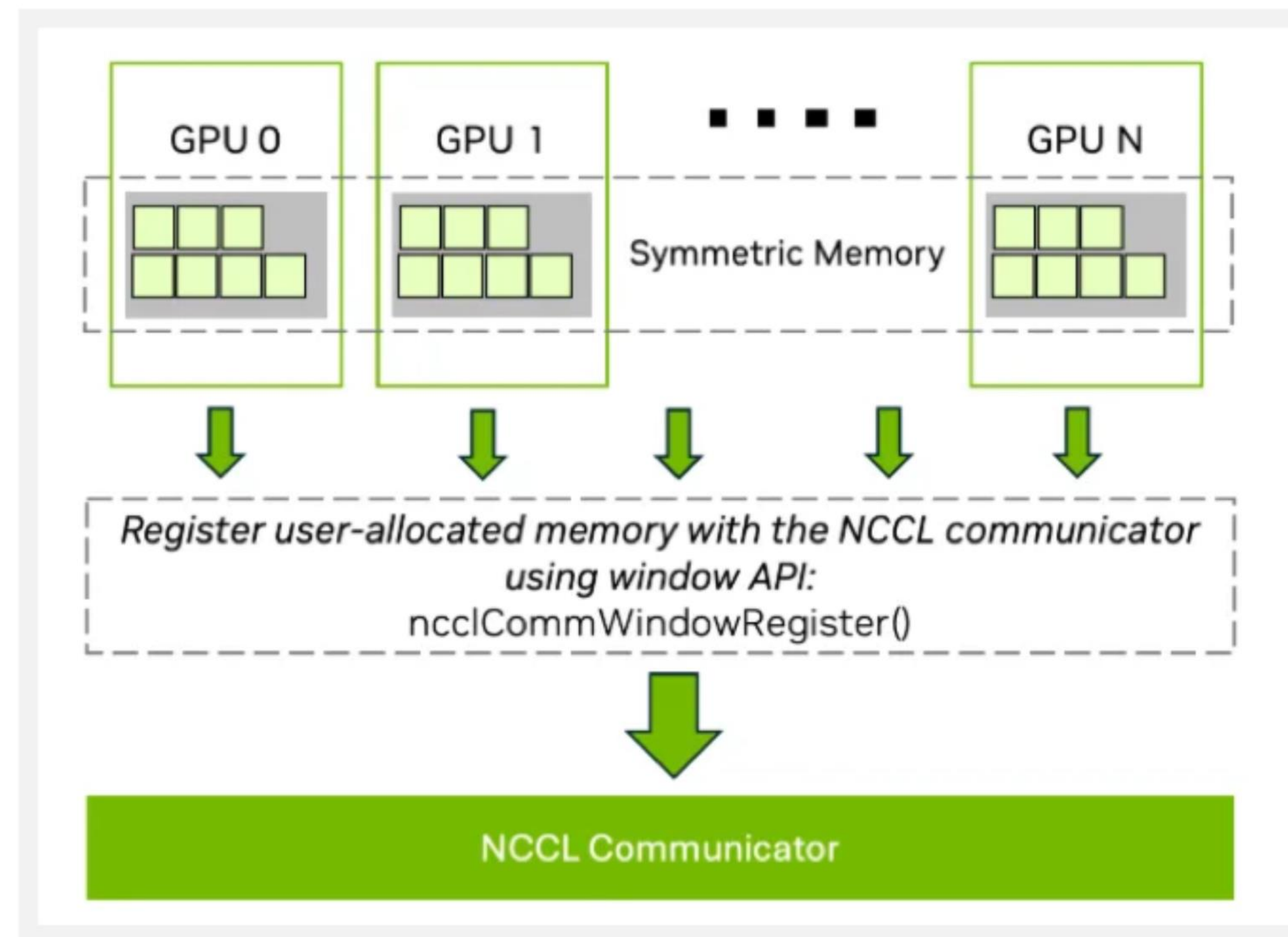
GPU-Initiated Networking (GIN)



A symmetric window exposes memory across many GPUs

The layout is common across ranks even when the CUDA pointer values differ

Without Symmetric:
Tag matching
Two-sided operation
Address translation

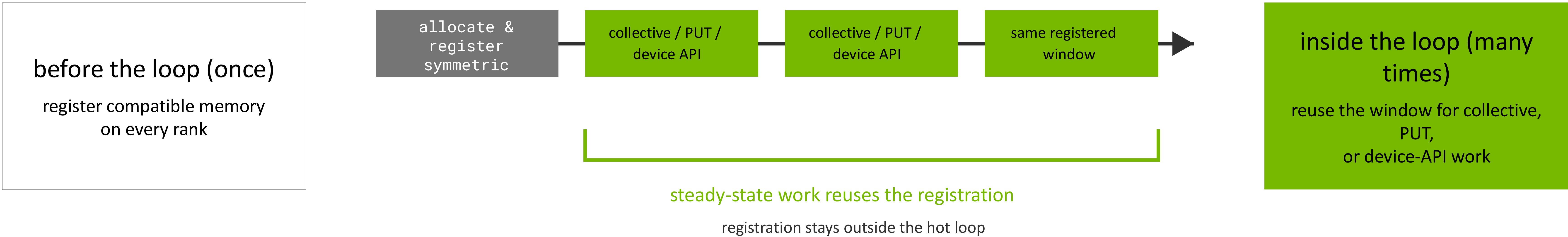


With Symmetric:
No tag matching
One-sided operation
No address translation

The resulting window is shared infrastructure for registered collectives, host PUT targets, and later device APIs.

Symmetric memory setup stays outside the hot loop

Pay setup cost once, reuse the window in every iteration



Register symmetric memory into a window

The collective stays an AllGather; the operands become registered windows

Register the source and destination, run the existing AllGather, then deregister only after the stream completes

Key APIs

```
ncclCommWindowRegister(comm, src, src_size, &src_win,  
                        NCCL_WIN_COLL_SYMMETRIC);  
ncclCommWindowRegister(comm, dst, dst_size, &dst_win,  
                        NCCL_WIN_COLL_SYMMETRIC);  
ncclAllGather(src, dst, src_size, ncclInt8, comm, stream);
```

After `cudaStreamSynchronize(stream)`, deregister both windows before freeing the allocations.

Effect

- Registration is collective: every rank supplies compatible source and destination regions
- `ncclWindow_t` is a local opaque handle; the logical offsets match even when CUDA pointers differ
- AllGather semantics do not change: `dst` receives rank-ordered source segments

NCCL host PUT: one-sided, stream-ordered RMA

PUT writes a peer window; the signal hands off the payload

The target rank is not involved. The source names the peer, registered window and offset; the target queues a stream wait.

```
// source rank stream
ncclPutSignal(local, count, datatype,
              peer, dst_win, offset, 0, 0, 0, comm, stream);
// writes the peer window; its signal follows payload delivery

// destination rank stream
ncclWaitSignal(1, &wait, comm, stream);
// after completion, the matching payload is visible to later work
```

- Unlike NVSHMEM, All NCCL Puts come with signals
- You must pass a symmetric window into a NCCL Put in addition to an offset

Host PUT with symmetric operands

Key APIs

```
ncclPutSignal(device_send, kElements, ncclInt,  
             destination, recv_window, 0,  
             kSignalIndex, kContext, 0, comm, stream);  
ncclWaitSignalDesc_t wait = {1, source, kSignalIndex, kContext};  
ncclWaitSignal(1, &wait, comm, stream);
```

Effect

- Destination does not post ncclRecv; PUT writes its registered recv_window and then updates its remote signal
- When ncclWaitSignal completes, the matching payload is visible to later work on this stream
- CPU return only means queued; the source is reusable after its stream reaches the PUT

In this lab sigIdx = 0, ctx = 0, and flags = 0

The target window is registered before the PUT

Device SIGNAL and PUT with symmetric operands

Key APIs

```
ncclGin gin{dev_comm, 0};

// synchronization

gin.signal(team, // team object for group of peers
           peer,  // rank of communication target
           ncclGin_SignalInc{(ncclGinSignal_t)0}, // ++signal[0]
           ncclCoopThread() ); // one thread calls this

gin.waitSignal(ncclCoopThread(), // one thread calls this
               (ncclGinSignal_t)0, // signal[0]
               value); // desired value of signal

// data movement

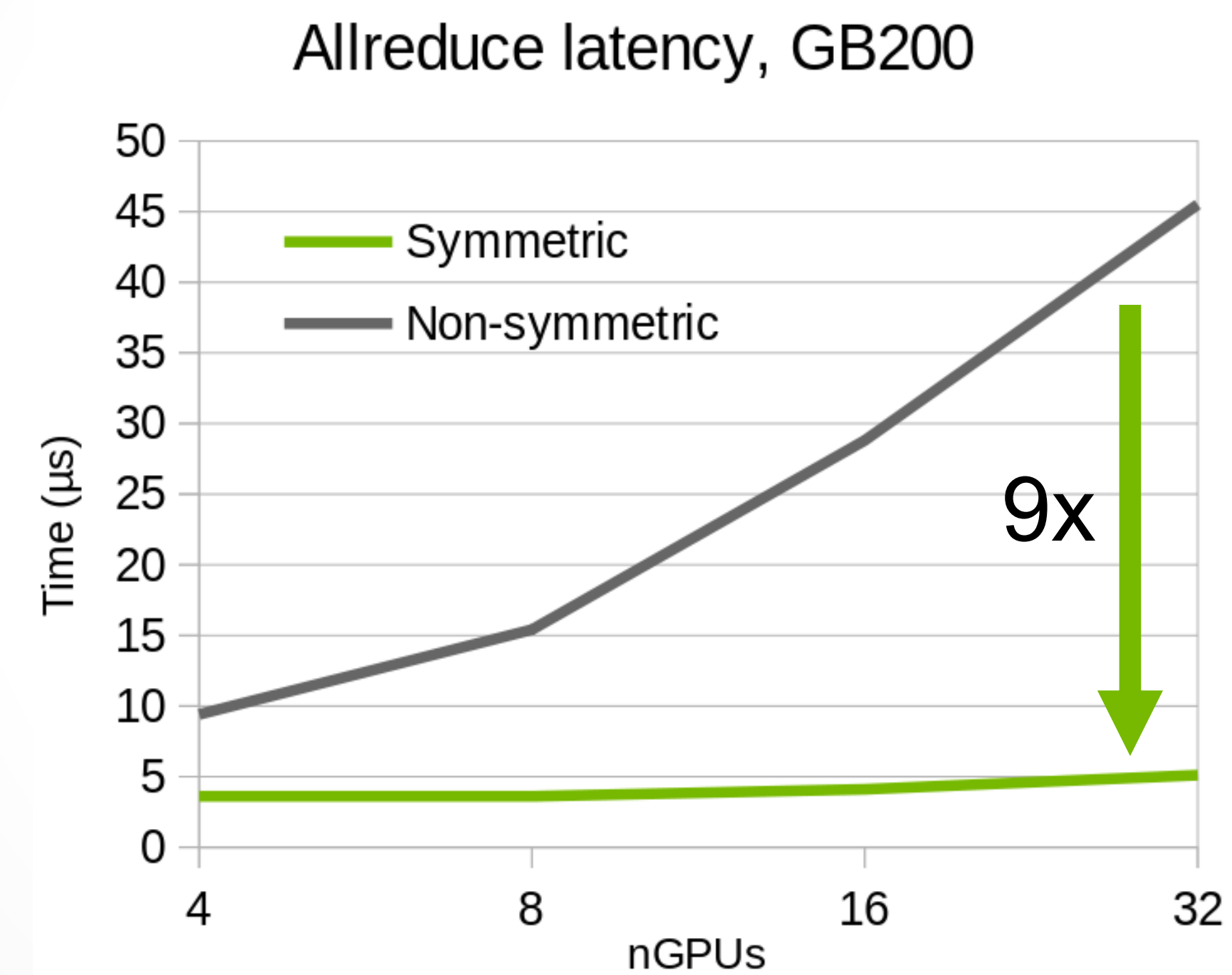
void put(ncclTeam team, int peer, // see above
         ncclWindow_t dstWnd, size_t dstOffset, // target memory location
         ncclWindow_t srcWnd, size_t srcOffset, size_t bytes, // source memory location and size
         RemoteAction remoteAction, LocalAction localAction, // signal actions (OPTIONAL)
         Coop coop, // thread, warp or block scope for this call (like NVSHMEM)
         DescriptorSmem descriptor, // optional SMEM scratch
         cuda::thread_scope alreadyReleased, cuda::thread_scope expected_scope, // memory consistency control
         SegmentType bufType) // use default unless window VA spans host memory
```

Conclusion



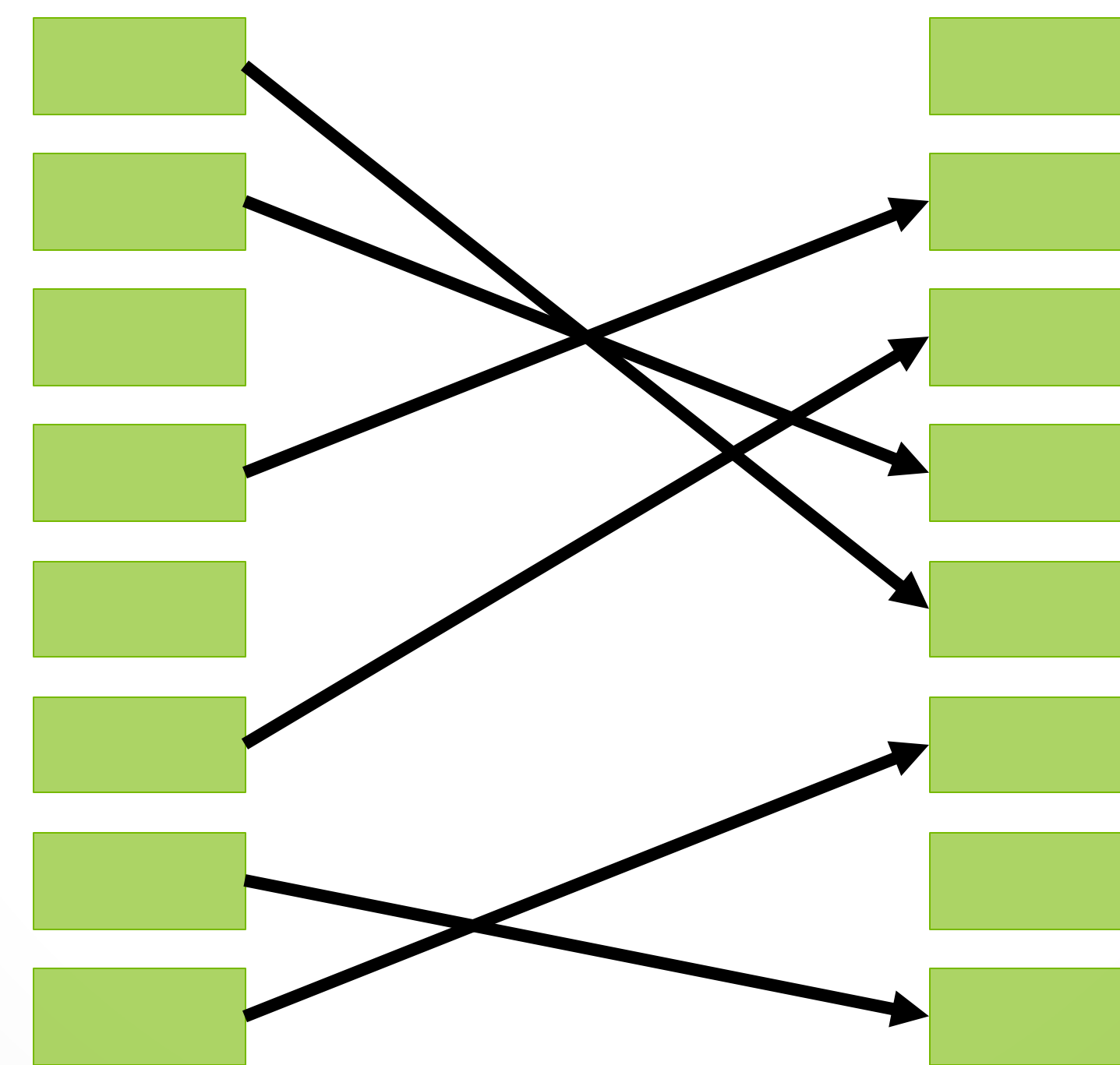
Today's Challenges in Distributed Workloads

Latency-sensitive operations (e.g. Inference)



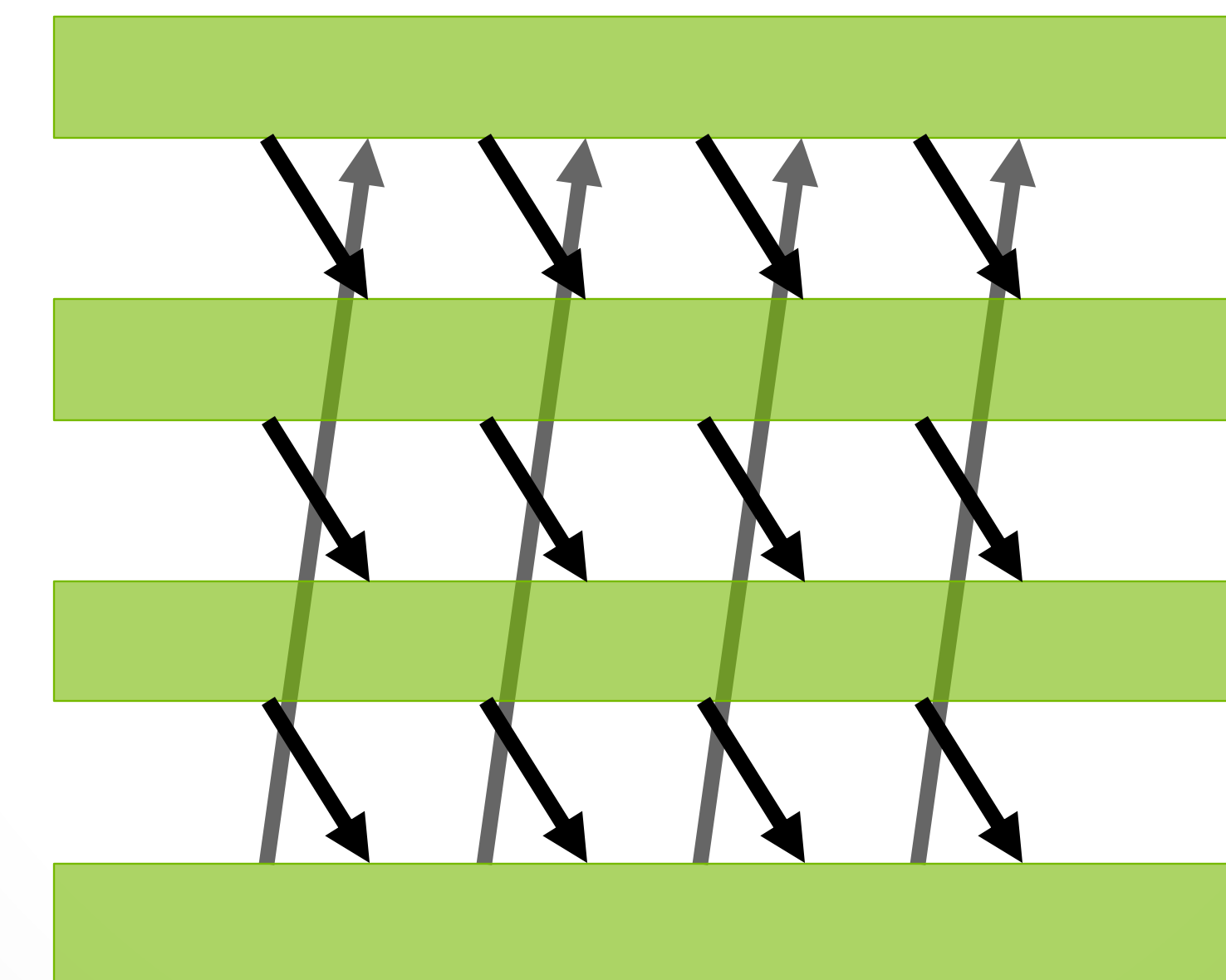
vLLM, SGLang, TRT-LLM

Custom comm patterns (e.g. MoE)



On-device metadata
(dest IDs, splits)

Comm-compute fusion In-kernel communication



Flash-infer, Helion

Additional Resources

GTC: *Multi-GPU Communication Libraries for Scaling HPC and AI Workloads*

<https://www.youtube.com/watch?v=kyQtbyR536I>

HotInterconnects Tutorial: *GPU Communication Libraries for Accelerating HPC and AI Applications*

<https://www.youtube.com/watch?v=rlA5QreHekk>

GPU MODE Lecture 67: *NCCL and NVSHMEM*

<https://www.youtube.com/watch?v=zxGVvMN6WaM>

GPU MODE Lecture 113: *Every Microsecond Matters: Achieving Near Speed-of-Light Latency in GPU Collectives*

<https://www.youtube.com/watch?v=J7-uvBSG7ho>